

Operating systems

Mutex, Semaphore

Content Lecture#9

01	Mutex
02	Semaphore
03	C
04	S
05	I

Lets take a Break

A programmer had a problem, so he used threads. Then he had two problems.







Mutex Locks

The hardware-based solutions to the critical-section problem

Semaphore

- In 1965, Dijkstra (Dutch Scientist) invented synchronize variable that takes non negative integer variable

flag[2] = {0,0};

```
Do    //thread#1
{
flag[0]=true;
while(flag[1]);

        critical section

flag[0]=false;

        remainder section
} while(1)
```

```
Do //Thread#2
{
flag[1]=true;
while(flag[0]);

        critical section

flag[1]=false;

        remainder section
} while(1)
```

turn = 0;

```
while(turn != 0);  //entry section

//critical section

turn = 1; //exit section

//remainder section<-==========
```

```
while(turn != 1);//entry section

//critical section

turn = 0;//exit section

//remainder section
```

flag[2] = {0,0}; turn = 0;

```
Do  //flag[0] turn=0  //thread#
{
flag[0]=true;
turn=1;
while(flag[1] && turn==1){  }

      critical section

flag[0]=false;

      remainder section
} while(1)
```

```
Do   //flag[1] turn=1
{
flag[1]=true;
turn=0;
while(flag[0] && turn==0);

      critical section

flag[1]=false;

remainder section
} while(1)
```

Mutex Locks

The hardware-based solutions to the critical-section problem
- are complicated.
- inaccessible to application programmers.

Operating-systems designers build software tools called **mutex lock**.

mutex lock to protect critical regions and thus prevent race conditions.
Process must acquire the lock before entering a critical section. "**acqu**
Process releases the lock when it exits the critical section. **"release()"**

```
acquire()
{
  while (!available) ;
                /* busy wait */
 available = false;;
}
```

```
release()
 {
 available = true;
}
```

* Calls to either acquire() or release() must be performed atomically

```
do {
        acquire lock
            critical section
        release lock
            remainder section
} while (true);
```

Critical section solution using mutex lock

A mutex lock has a boolean variable available whose value indicates if the lock is available or not. If the lock is available, a call to acquire() succeeds, and the lock is then considered unavailable. A process that attempts to acquire an unavailable lock is blocked until the lock is released. The definition of acquire() is as follows:

Semaphores

- In 1965, Dijkstra (Dutch Scientist) invented synchronize variable that takes non negative integer variable
- A semaphore has two operations
  - Wait (**P**)
    - An atomic operation that waits for semaphore to become positive, then decrements its by 1
  - Signal (**V**)
    - An atomic operation that increments semaphore by 1 after completing operation and releasing semaphore
- Entry to critical sections of active processes is controlled by the **P** operation and exit is controlled by **V** operation
- A semaphore such as **S,** indicates the availability of some resource;
  - If it has a non-zero positive value, it is available
  - If it is zero, it is unavailable, i.e. it is being accessed by another process

Semaphore

- The wait operation allows a process to access a resource (S>0) or may cause the process to be blocked
- The signal operation signals that some process has released resource and now it can be used by a process waiting for
- These operations are used to bracket the critical sections of the process;

```
wait(S)
        <critical section>
signal(S)
```

- Assume **S** is initially set to value 1 (resource is available)
  - The first process to execute the wait will set **S** to **S-1** and will enter to its critical section
  - If another process reaches its critical section, the wait now finds **S** set to 0 and process is blocked
  - When first process is exit from its critical section, it executes the signal, **S** would set to 1

Semaphore

- For each semaphore, the system must maintain a queue of waiting processes
- The semaphore operations are implemented as OS services
- The OS will guarantee that wait and signal are invisible
  - i.e. once started they cannot be interrupted
- The semaphore with values 1 or 0 are termed as binary semaphore
- Counting semaphore may have any non-negative value
  - The value indicate the available quantity of a particular resource type

Semaphore Properties

- Attractive Properties
  - Machine independent
  - Simple
  - Work with many
  - For more critical section use different semaphores
  - Acquire many resources simultaneously
  - Permit multiple processes into critical section at once if desirable
- Limits of semaphores
  - Semaphores are not provided by hardware
  - Programmers must use them correctly
  - Cannot test busy without blocking
  - Indefinite blocking

Semaphore-Examples

- Producer Consumer problem
  - A producer process generates data and store them in the next available buffer in array
  - Process consumer is removing items from buffer and processing them
  - Assume number of positions in the buffer is finite, say **N**
  - A particular example is print routine, which is sending characters to a printer driver for printing
  - We need to protect buffer since both process can access it
  - The additional problems are;
    - The producer must wait for an empty space if buffer is full
    - The consumer must wait for data available if buffer is empty

Semaphores : Producer Consumer

Semaphore	**Purpose**	**Initial value**
Free Space Data	Mutual exclusion for buffer access Space available in buffer Data available in buffer	1 N 0
Producer	**Explanation**	
Produce item wait(space) wait(free) add item to buffer	Application produces data item If buffer full, wait for space signal If buffer being used, wait for free signal Put item in next buffer slot Signal that buffer no longer in use	

Semaphores : Producer Consumer

Consumer	Explanation
wait(data) wait(free) get item from buffer signal(free) Signal(space) Consume item	Wait until at least one item in buffer If buffer being used, wait for free signal Get item from buffer Signal that buffer no longer in use Signal that at least one space exist in buffer Application specific processing of item

The counting semaphore **data** and **space** are also used as counters to monitor the occupancy of buffer

Wait and signal operations decrement and increment these counters

The producer's signal (data) will increment the count of the number of data item

Semaphore: Unix

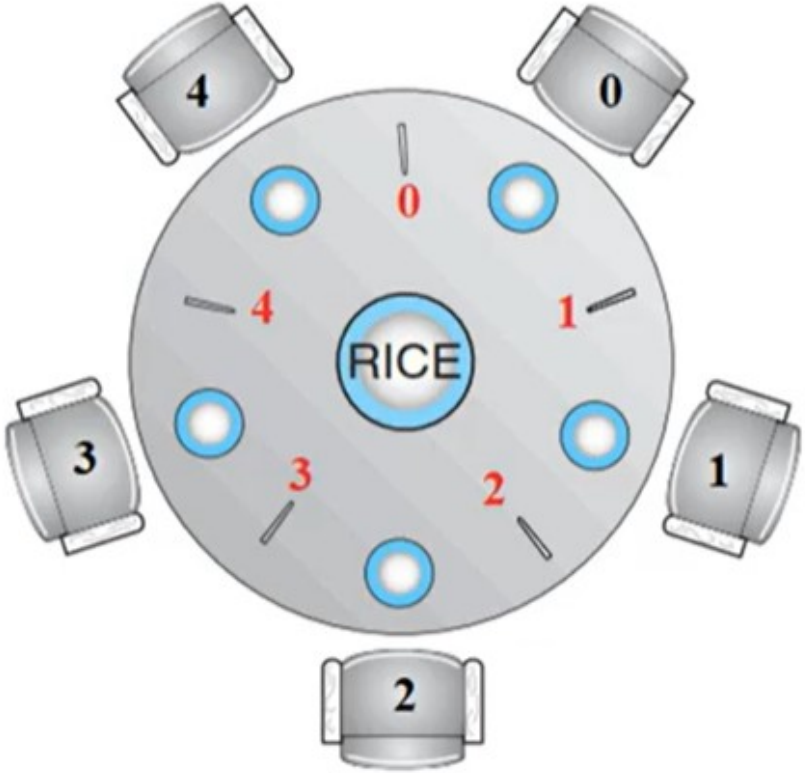
- Unix provide facility for creation and handling of semaphores
- The **semget** function call is used in a program to create semaphore variables, it creates an array of semaphores
- The semget call specifies
  - The number of semaphores required
  - A unique key value to identify semaphore
  - Some access and control information
- The function call **semctrl** provides a range of semaphore maintenance services; such as
  - Examining the value of a semaphore
  - Removing a semaphore set, and
  - Initializing the semaphore
- The detailed processing of the semaphore is carried out by **semop** function call

Semaphore vs Mutex

Parameters	Semaphore	Mutex
Mechanism	It is a type of signaling mechanism.	It is a locking mechanism.
Data Type	Semaphore is an integer variable.	Mutex is just an object.
Modification	The wait and signal operations can modify a semaphore.	It is modified only by the process that may request or release a resource.
Resource management	If no resource is free, then the process requires a resource that should execute wait operation. It should wait until the count of the semaphore is greater than 0.	If it is locked, the process has to wait. The process should be kept in a queue. This needs to be accessed only when the mutex is unlocked.
Thread	You can have multiple program threads.	You can have multiple program threads in mutex but not simultaneously.
Ownership	Value can be changed by any process releasing or obtaining the resource.	Object lock is released only by the process, which has obtained the lock on it.
Types	Types of Semaphore are counting semaphore and binary semaphore.	Mutex has no subtypes.
Operation	Semaphore value is modified using wait () and signal () operation.	Mutex object is locked or unlocked.
Resources Occupancy	It is occupied if all resources are being used and the process requesting for resource performs wait () operation and blocks itself until semaphore count becomes >1.	In case if the object is already locked, the process requesting resources waits and is queued by the system before lock is released.

Dining Philosophers Problem

- Philosophers spend their lives thinking and eating
- Don’t interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers
  - Shared data
    - Bowl of rice (data set)
    - Semaphore chopstick [5]



Semaphores

A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations:

- wait() “to test”
- signal() “to increment”

```
wait(S)
{
while (S <= 0) ;
        // busy wait S- -;
}
```

definition of wait()

```
signal(S)
{
 S++;
 }
```

definition of signal()

All modifications to the integer value of the semaphore in the wait() and signal() operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Mutex vs Semaphore

Mutex used for mutual exclusion

acquire(&Lock0);

release(&Lock);

Semaphore

Wait(&Lock0);

signal(&Lock);

Binary semaphore lock=1 mutex lock
Counting semaphore lock=1,2,3,…N

Mutex & Semaphore

```
data;
mutex = 1;
read()
{
        acquire(&mutex); or wait(&mutex);
        cout<<data;
        release (&mutex) or simple signal(&mutex);
}
write()
{
        acquire(&mutex); or wait(&mutex);
        data=getdata();
        release (&mutex) or simple signal(&mutex);
}
```

Reader-Writer Algorithm

(Reader priority and writer thread starves)

- what if there are multiple readers why should they enter one by one and data is not being effected..
- But read and write can not enter at the same time as read operation is shared.

- Reader-Writer Algorithm -1 Reader priority and writer thread starves
- Reader-Writer Algorithm -2 Writer priority and Reader thread starves
- Reader-Writer Algorithm -3 Writer enter simultaneously and then readers

```
multiple readers can enter in critical section

rw_mutex_lock =1;
count =0;
mutex=1
read
{
        wait(&mutex);
        count++;
        signal(&mutex);
        //--> contex switch of time slice expires
        if(count==1)
                write(&rw_mutex_lock);
                //critical section
        wait(&mutex);
        count--;
        signal(&mutex);
        if(count==0)
                signal(&rw_mutex_lock);

}
Write()
{
        Wait ()&rw_mutex;
        data =getdata();
        signal(&rw_mutex_Lock)

}
```

Producer-Consumer Problem

Inter process communication is done through PIPE()
Inter thread communication is done using a buffer with any size but need to incorporate check PIPE ican be used but avoided for performance due to system call invoke

Producer only produce data where buffer has some space

If buffer is empty consumer blcok

```
buffer =[10];
count=0;     /// to chekc if there is space available in buffer
in =0; out =0; mutex=1;
producer() //sender
{
        while (count--10){}
        buffer [in] = getdata();
        in =(in + 1)%10;
        wait(&mutex);
        count++;
        signal(&mutex);
}
Consumer() //receiver{
        while(count==0){}
        cout<<buffer{out};
        out = (out+1)%10;
        wait(&mutex);
        count--;
        signal(&mutex);
}
```

Bounded Buffer problem

```
buffer =[10];
in =0;
out =0;
empty =10;
full =0;

producer() //sender
{
                wait(&empty);
                buffer [in] = getdata();
                in =(in + 1)%10;
                signal(&full);


}
Consumer() //receiver
{
                wait(&full);
                cout<<buffer{out};
                out = (out+1)%10;

                signal wait(&empty);

}
```

Dining Philosopher

```
semaphore chopstick[5];
5.8 Monitors 223
do {
wait(chopstick[i]);
wait(chopstick[(i+1) % 5]);
...
/* eat for awhile */
...
signal(chopstick[i]);
signal(chopstick[(i+1) % 5]);
...
/* think for awhile */
...
} while (true);
```

H2O

There are two kinds of threads, oxygen and hydrogen. In order to assemble these threads into water molecules, we have to create a barrier that makes each thread wait until a complete molecule is ready to proceed. As each thread passes the barrier, it should invoke bond. You must guarantee that all the threads from one molecule invoke bond before any of the threads from the next molecule do. In other words:

- If an oxygen thread arrives at the barrier when no hydrogen threads are present, it has to wait for two hydrogen threads.
- If a hydrogen thread arrives at the barrier when no other threads are present, it has to wait for an oxygen thread and another hydrogen thread.

```
Semaphore mutex = 1;
Semaphore h_wait = 0;
Semaphore o_wait = 0;
int count = 0;

hReady() {
    P(mutex);
    count++;
    if(count %2 == 1) {
        V(mutex);
        P(h_wait);
    } else {
        V(o_wait);
        P(h_wait);
        V(mutex);
    }

    return;
}
```

```
oReady()
{
    P(o_wait);
    V(h_wait);
    V(h_wait);
    makeWater();

    return;
}
```

Condition Variable

Semaphore

Semaphore, as name suggests, is basically an object that includes counter, waiting list of process and supports two different operations i.e., wait and signal. Its type includes counting semaphores and binary semaphores. It is simply a synchronization tool that can be used to deal with critical-section problem. It provides simple abstraction for controlling access to a common resource in programming environment.

Condition Variable

Condition Variable, as name suggests, is simply a synchronization primitive that allows threads to wait until particular condition occurs. It includes two operations i.e., wait and signal. It just allows thread to be signaled when something of interest to that threads occurs and is mostly used when one wants to know when something happens.

Condition Variable

Semaphore	Condition Variable
It does not allow threads to wait. Instead, each thread keeps running and last thread that will set semaphore value to zero will go to sleep.	It allows threads to wait until particular condition occurs.
It is generally used to solve problem of some critical sections in process synchronization.	It is generally used with mutex to signal changing states from one thread to another one.
Its main aim is to control access to common resource by multiple processes and avoid critical section problems in concurrent system like multitasking operating system.	Its main aim is to support operations that wake one or wake all waiting threads.
It can be used anywhere except in a monitor.	It can only be used in monitors.
In this, wait () does not always block its caller.	In this, wait () usually blocks its caller always.
They are sticky as they have memory and sem_post() will increment semaphore even if no one has called sem_wait().	They are non-sticky as signal () is not saved if there is no one waiting for signal ().
It is simply a counter + mutex + wait queue.	It is simply a wait-queue.
It can be used as a conditional variable and therefore is more lightweight and flexible.	It cannot be used as a semaphore and therefore is not flexible.
In this, we cannot unlock all waiting threads in same instant using broadcast unlocking.	In this, we can unlock all waiting threads in same instant using broadcast unlocking.
Signals are not lost even if there is nobody waiting on the queue.	Signals are lost if there is nobody waiting on the queue.

Deadlock Avoidance

Requires that the system has some additional ***a priori*** information available

- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

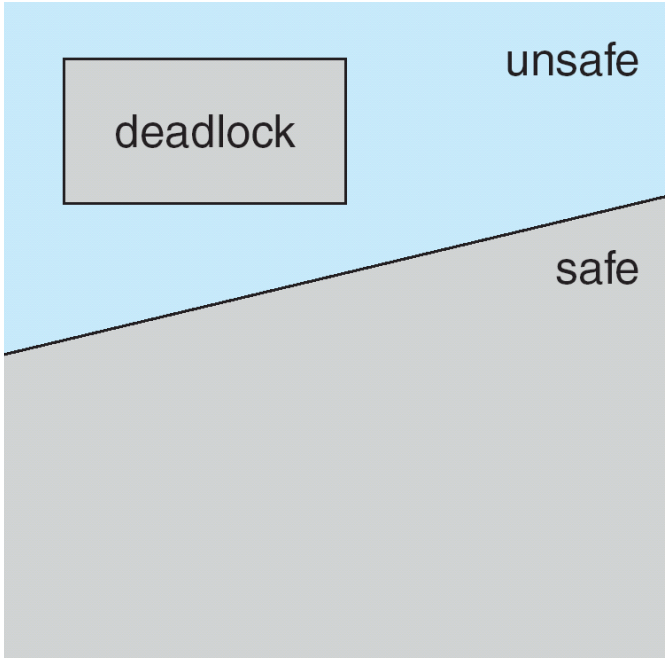
Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in **safe state** if there exists a sequence <P_1, P_2, …, P_n> of ALL the processes in the systems such that for each P_i, the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < I$
- That is:
  - If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
  - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
  - When P_i terminates, P_{i+1} can obtain its needed resources, and so on

Basic Facts

- If a system is in safe state ⇒ no deadlocks
- If a system is in unsafe state ⇒ possibility of deadlock
- Avoidance ⇒ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State



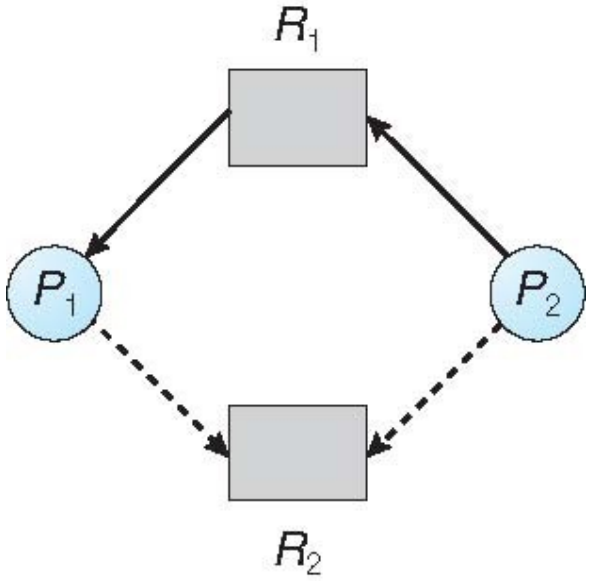
Avoidance Algorithms

- Single instance of a resource type
  - Use a resource-allocation graph
- Multiple instances of a resource type
  - Use the banker’s algorithm

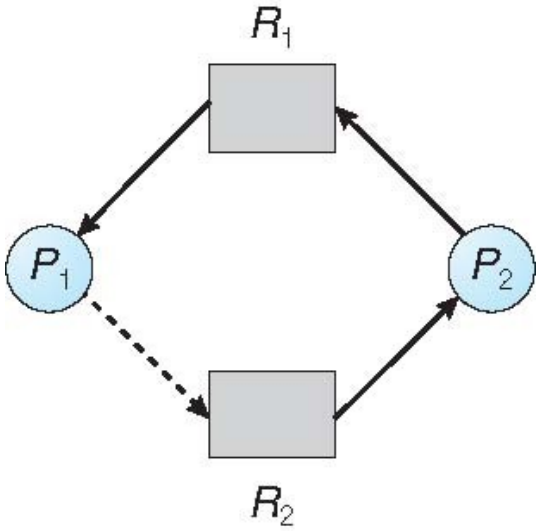
Resource-Allocation Graph Scheme

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system

Resource-Allocation Graph



Unsafe State In Resource-Allocation Graph



Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

Banker’s Algorithm

- Multiple instances
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available***:* Vector of length m. If available [j] = k, there are k instances of resource type R_j available

- **Max***: n x m* matrix. If *Max* [i,j] = k, then process P_i may request at most k instances of resource type R_j

- **Allocation***:* n x m matrix. If Allocation[i,j] = k then P_i is currently allocated k instances of R_j

- **Need***:* n x m matrix. If *Need*[i,j] = k, then P_i may need k more instances of R_j to complete its task

Safety Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:
   Work* = *Available
   ***Finish* [*i*] = *false* for *i* = 0, 1, ..., *n*- 1**

2. Find an ***i*** such that both:
   (a) ***Finish* [*i*] = *false***
   (b) $\boldsymbol{Need_i \leq Work}$
   If no such ***i*** exists, go to step 4

3. $\boldsymbol{Work = Work + Allocation_i}$
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state

Resource-Request Algorithm for Process P_i

Request$_i$ = request vector for process ***P***$_i$. If ***Request***$_i$**[*j*] = *k*** then process ***P***$_i$ wants ***k*** instances of resource type ***R***$_j$

1. If ***Request***$_i \leq$ ***Need***$_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If ***Request***$_i \leq$ ***Available***, go to step 3. Otherwise ***P***$_i$ must wait, since resources are not available
3. Pretend to allocate requested resources to ***P***$_i$ by modifying the state as follows:

$$\textbf{\textit{Available}} = \textbf{\textit{Available}} - \textbf{\textit{Request}}_i;$$

$$\textbf{\textit{Allocation}}_i = \textbf{\textit{Allocation}}_i + \textbf{\textit{Request}}_i;$$

$$\textbf{\textit{Need}}_i = \textbf{\textit{Need}}_i - \textbf{\textit{Request}}_i;$$

- If safe ⇒ the resources are allocated to ***P***$_i$
- If unsafe ⇒ ***P***$_i$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

- 5 processes P_0 through P_4;
  3 resource types:
  A (10 instances), *B* (5instances), and *C* (7 instances)
- Snapshot at time T_0:

	Max	*Available*	*Allocation*
	A B C	*A B C*	*A B C*
P_0	7 5 3	3 3 2	0 1 0
P_1	3 2 2		2 0 0
P_2	9 0 2		3 0 2
P_3	2 2 2		2 1 1
P_4	4 3 3		0 0 2

Example (Cont.)

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	Need
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

- The system is in a safe state since the sequence < P_1, P_3, P_4, P_2, P_0> satisfies safety criteria

Example: P_1 Request (1,0,2)

- Check that Request ≤ Available (that is, (1,0,2) ≤ (3,3,2) ⇒ true

	Need	*Available*	*Allocation*
	A B C	*A B C*	*A B C*
P_0	7 4 3	2 3 0	0 1 0
P_1	0 2 0		3 0 2
P_2	6 0 0		3 0 2
P_3	0 1 1		2 1 1
P_4	4 3 1		0 0 2

- Executing safety algorithm shows that sequence < $\boldsymbol{P_1}$, $\boldsymbol{P_3}$, $\boldsymbol{P_4}$, $\boldsymbol{P_0}$, $\boldsymbol{P_2}$> satisfies safety requirement
- Can request for (3,3,0) by $\boldsymbol{P_4}$ be granted?
- Can request for (0,2,0) by $\boldsymbol{P_0}$ be granted?



Thank you